MIESC: A Multi-layer Framework for Automated Smart Contract Security Evaluation

Fernando Boiero
Universidad Tecnológica Nacional (UTN)
Facultad Regional Villa María
Villa María, Córdoba, Argentina
fboiero@frvm.utn.edu.ar

***Abstract*—Automated smart contract security tools each capture only a fraction of existing vulnerabilities. We present MIESC, an open-source framework that orchestrates 35 analysis modules across 9 defense layers—static analysis, dynamic testing, symbolic execution, formal verification, AI/LLM analysis, pattern detection, DeFi-specific analysis, exploit validation, and consensus—with RAG-enhanced false-positive filtering, an intelligence engine for cross-tool Bayesian confidence scoring, and automated remediation (). On SmartBugs-curated (143 contracts), MIESC achieves 80% recall vs. Slither's 43%. On 11 real DeFi exploits ($3.3B losses), recall reaches 81.8% (κ=0.77). On EVMBench (OpenAI/Paradigm; 40 real audits, 120 business-logic vulnerabilities), MIESC with Claude Sonnet achieves 82.5% recall on the full benchmark ($5.50 total) and 94.1% on a 10-audit subset—surpassing Claude Opus standalone (45.9%) by 37 percentage points. Critically, a fully local 32B-parameter model achieves 82.4% recall at zero cost, outperforming GPT-4o API (64.7%). These results demonstrate that *orchestrated static analysis combined with any LLM is strictly superior to even the best LLM alone*. MIESC is available under AGPL-3.0.**

***Index Terms*—smart contracts, blockchain security, vulnerability detection, defense-in-depth, multi-agent systems, retrieval-augmented generation, static analysis, formal verification**

I. Introduction

Smart contracts govern over $100 billion in decentralized finance (DeFi) protocols, yet vulnerabilities continue to cause catastrophic losses: the Wormhole bridge exploit ($320M, 2022), the Euler Finance hack ($197M, 2023), and the Curve Finance reentrancy attack ($70M, 2023) underscore the urgency of rigorous security analysis before deployment [1].

Existing automated tools—Slither [2], Mythril [3], Echidna [4]—each implement a single analysis technique and capture only a subset of vulnerabilities. Durieux et al. [5] evaluated 9 tools on 47,587 contracts and found that no single tool achieved both high precision and high recall. In practice, a security researcher preparing a pre-audit report runs 5–10 tools independently, normalizes their heterogeneous outputs by hand, and cross-references findings to filter duplicates—a workflow that takes 4–8 hours per contract and does not scale to the thousands of contracts deployed weekly on Ethereum and its Layer-2 networks.

We address this gap with MIESC, a framework that:

1) Orchestrates **13 external security tools** and **22 internal analysis modules** across **9 complementary defense layers**, each targeting a distinct class of vulnerabilities.
2) Normalizes heterogeneous tool outputs into a unified finding schema with SWC/CWE classification.
3) Filters false positives using a **RAG-enhanced ML pipeline** with 60 curated vulnerability patterns and per-detector calibration.
4) Maps findings to **12 international security standards** (ISO 27001, NIST CSF, OWASP, CWE, SWC, MITRE ATT&CK).
5) Provides an **extensible plugin system** for community-driven detector development.
6) Implements an **agentic deep-audit loop** that, on CRITICAL findings, queries a primary code-analysis model plus a dedicated verification-role model and applies bounded confidence adjustments ($+0.20$ on agreement, -0.30 on joint rejection, -0.10 + manual-review flag on disagreement)—turning the LLM into a calibrated verifier rather than a single-shot labeler.
7) Closes the **specification-to-proof loop** by generating Certora CVL rules, Scribble annotations, and SMTChecker assertions directly from findings, and exposing a unified command that runs the generated specs against the Certora Prover, Halmos, and solc's SMTChecker.
8) Extends coverage to the **Starknet / Cairo ecosystem** with 13 vulnerability classes informed by 2024–2026 real-world exploits (zkLend stale-price, Braavos account re-initialization, Cairo 1.0 unchecked arithmetic, dropped syscall results, account-abstraction signature replay).

MIESC is designed as a *pre-audit triage tool*, not a replacement for manual expert review. By automating the multi-tool orchestration and normalization workflow, it reduces audit preparation time and enables continuous security monitoring in CI/CD pipelines.

II. Related Work

A. Single-Technique Analysis Tools

Static analysis. Slither [2] uses intermediate representation (SlithIR) to detect vulnerability patterns with low false positive rates. Aderyn [6] implements Rust-based AST analysis for high performance. Solhint provides Solidity-specific linting rules.

Dynamic testing. Echidna [4] performs property-based fuzzing using coverage-guided input generation. Foundry's forge tool combines unit testing with fuzz-based invariant checking. Medusa extends fuzzing with corpus-based strategies.

Symbolic execution. Mythril [3] explores execution paths using SMT solving to find reachable vulnerabilities. Halmos [7] enables symbolic testing within the Foundry framework. Manticore [8] combines symbolic execution with concrete execution.

Formal verification. Certora Prover verifies user-specified properties using SMT-based techniques. SMTChecker is Solidity's built-in formal verification engine.

B. Multi-Tool Frameworks

SmartBugs [9] orchestrates 19 analysis tools with Docker-based execution and SARIF output. It serves primarily as an academic benchmarking framework. SmartBugs 2.0 [10] added parallel execution and improved tool integration.

MIESC differs from SmartBugs in three key aspects: (1) it integrates 35 analysis modules (13 external + 22 internal) across 9 layers vs. 25 tools, (2) it adds AI/LLM semantic analysis and RAG-enhanced false positive filtering—capabilities absent from SmartBugs, and (3) it produces production-ready outputs (professional reports, SARIF, GitHub Action) for CI/CD integration.

C. AI-Enhanced Security Analysis

GPTScan [11] combines GPT-4 with static analysis for logic vulnerability detection, achieving results that complement pattern-based tools. Other recent approaches apply local LLMs for multi-agent auditing (LLMSmartAudit) and RAG-enhanced code analysis (SmartLLM). These tools demonstrate the potential of AI-assisted auditing but operate as standalone analyzers. MIESC integrates them as one layer within a broader multi-technique pipeline, using their outputs alongside static, dynamic, and symbolic results for cross-validated findings.

III. Architecture

MIESC implements a *defense-in-depth* architecture organized into 9 layers, following the principle that multiple independent security mechanisms provide stronger protection than any single mechanism [12]. Each layer targets complementary vulnerability classes (Table I).

A. Orchestration Pipeline

The analysis pipeline proceeds as follows:

1) **Input**: Solidity source code (or Vyper, Rust, Move for non-EVM chains).
2) **Tool Execution**: Each layer runs its tools in parallel with configurable timeouts. An adapter pattern normalizes tool-specific outputs into a unified schema.
3) **Aggregation**: The Finding Aggregator deduplicates results using SWC/CWE classification and source location matching.

TABLE I
MIESC'S 9 DEFENSE LAYERS

Layer	Technique	Ext.	Int.
1	Static Analysis	4	–
2	Dynamic Testing	3	–
3	Symbolic Execution	3	–
4	Formal Verification	3	1
5	AI/LLM Analysis	–	6
6	Pattern Detection	–	5
7	DeFi Security	–	4
8	Exploit Validation	–	3
9	Consensus & Reporting	–	3
	Total	**13**	**22**

Ext. = third-party tools invoked as separate processes. Int. = built-in analysis modules (LLM, pattern matching, consensus).

4) **ML Pipeline**: A false positive filter applies per-detector calibration rates, context analysis (guards, CEI patterns, OpenZeppelin usage), and Solidity version awareness.
5) **RAG Enhancement**: A Retrieval-Augmented Generation system (Section III-B) enriches findings with vulnerability context from a curated knowledge base.
6) **Report Generation**: Findings are output as JSON, SARIF (for GitHub Security integration), PDF (professional audit reports), or Markdown.

B. RAG System

The RAG component uses ChromaDB as a vector store with sentence embeddings (all-MiniLM-L6-v2, 384 dimensions). It indexes 60 curated vulnerability patterns organized by category:

- **Reentrancy** (6 patterns): classic, cross-function, read-only, Vyper-specific
- **MEV** (4): sandwich attacks, JIT liquidity, time-bandit
- **Governance** (4): flash loan voting, timelock bypass
- **Proxy** (4): storage collision, uninitialized proxy
- **Bridge/Cross-chain** (3): message replay, oracle manipulation
- **ZK/NFT/DeFi** (6): circuit constraints, royalty bypass, oracle manipulation
- **SWC patterns** (33): covering SWC-100 through SWC-136 (unchecked calls, integer overflow, tx.origin, delegatecall, etc.)

Each pattern includes the vulnerability description, severity, a real-world exploit reference (e.g., Curve $70M, Euler $197M, Ronin $624M), structured attack steps (an ordered list of exploitation actions an attacker would take), a grep-level detection heuristic (specific code constructs the tool should look for), and fix templates. The attack steps and detection heuristics are surfaced verbatim in the LLM context block, giving the model an actionable procedure rather than a prose description. All 60 patterns carry a reference; the top 8 patterns—selected by exploit frequency—additionally carry explicit and fields. The system uses $O(1)$ lookup with an LRU cache (256 entries, 5-minute TTL) achieving 90%+ cache hit rate in batch analysis.

C. False Positive Filter

The FP filter employs a multi-signal approach:

- **Per-detector FP rates**: Empirically calibrated rates for 17+ detectors (e.g., Slither reentrancy: 15%, Mythril integer overflow: 8%).
- **Context analysis**: Detection of reentrancy guards, Checks-Effects-Interactions patterns, and OpenZeppelin/Solmate library usage.
- **Solidity version awareness**: Suppression of integer overflow warnings for Solidity $\geq$ 0.8.0 (built-in overflow protection).
- **RAG validation**: Cross-reference with curated vulnerability patterns for relevance scoring.

D. Agentic Deep Audit

Beyond batch orchestration, MIESC includes a *DeepAuditAgent* that implements an autonomous 4-phase investigation loop inspired by intelligent agent architectures [13]. Agents communicate via a Model Context Protocol (MCP) bus [14], enabling modular coordination:

1) **Reconnaissance**: Static call graph construction, taint analysis, risk profiling (DeFi, proxy, bridge, token), and attack surface scoring.
2) **Targeted Scan**: Adaptive tool selection based on risk profile—e.g., DeFi contracts trigger oracle and MEV detectors; proxy contracts trigger upgradability checkers.
3) **Deep Investigation**: Iterative agentic loop over HIGH/CRITICAL findings. Each finding triggers *type-specific* follow-up analysis: oracle/price findings invoke a DeFi pattern detector scoped to the vulnerable function, access-control findings generate a Certora CVL rule targeting the function (Section III-E), and reentrancy/unchecked-call findings are confirmed by targeted taint analysis. RAG enrichment additionally validates whether a known mitigation is present in the code; when present, the finding is downgraded and marked .
4) **Synthesis**: Cross-finding correlation, exploit chain detection via graph traversal over the call graph, and optional LLM narrative generation (local-first via Ollama, with optional cloud API fallback).

a) Multi-LLM consensus for critical findings.: For every finding flagged as CRITICAL, Phase 3 runs a two-step LLM verification rather than a single-shot review. MIESC queries (i) the model configured for the use case (primary) and (ii) the model configured for the use case (verifier). When both models independently confirm the finding, the agent applies a $+0.20$ confidence boost and records . When both reject, the agent applies a -0.30 penalty and downgrades the severity from CRITICAL to HIGH. On disagreement, the agent applies a -0.10 penalty and raises a flag with the model-level justifications attached, surfacing contested findings for human triage. If the two use cases map to the same model (common in lightweight local deployments), the agent records and leaves confidence unchanged—distinguishing one-model agreement from genuine multi-model agreement. This mechanism is designed to reduce the two failure modes of single-LLM verification: over-confident acceptance of model hallucinations and silent dismissal of real findings.

This agentic approach reduces manual triage by automatically investigating and correlating the most critical findings, detecting multi-step exploit chains that single-pass analysis would miss, and explicitly flagging the subset of findings where automated verdicts disagree.

E. Formal Verification Bridge

A recurring friction point in smart-contract security work is the gap between *finding a bug* and *proving its absence after the fix*. MIESC narrows this gap with a two-command workflow:

- consumes a findings JSON and emits executable specifications in three formats: Certora CVL rules ({}), Scribble inline annotations (<), and SMTChecker assertions compatible with .
- executes those specifications against whichever prover is installed—the Certora Prover via , Halmos via its Foundry integration, or SMTChecker via —and normalizes the outputs into a unified (status, rules passed, rules failed, counterexamples, elapsed seconds). A Foundry-root autodetector walks upward from the contract to locate , allowing Halmos to run on projects with standard layouts without additional configuration.

The agentic Phase 3 loop (Section III-D) closes the inner cycle: when an access-control finding is produced, the agent automatically generates a CVL rule targeting the specific function and attaches it to the finding payload, ready for the operator to pipe into . This converts an informal finding into an actionable formal obligation in a single step, lowering the activation energy for formal verification in day-to-day audit work.

F. Multi-Chain Support

Although the SmartBugs-curated evaluation focuses on EVM/Solidity, MIESC's architecture treats the analysis chain as a first-class parameter. A dispatcher auto-detects the target language from the file extension and routes to the appropriate chain analyzer:

- **EVM** (,): the 35-module pipeline described above.
- **Starknet / Cairo** (): a purpose-built analyzer with 13 vulnerability types informed by real 2024–2026 exploits (zkLend $9.6M stale-price read in an empty market; Braavos account re-initialization; Cairo 1.0 unchecked arithmetic; dropped from / ; account-abstraction signature replay without nonce binding; and classical felt-overflow / L1↔L2 message / caller spoofing / storage collision / proxy-upgrade / reentrancy / access-control).
- **Move** () and **Solana / Anchor** (): scaffolded analyzers for Sui/Aptos and Solana ecosystems, currently surfaced for community contribution.

The chain-agnostic orchestration layer and the finding-normalization schema are shared across ecosystems, so pattern

detection, RAG enrichment, and multi-LLM consensus apply uniformly regardless of target chain.

IV. Evaluation

A. Experimental Setup

We evaluated MIESC on the SmartBugs-curated benchmark [5], the standard dataset for smart contract vulnerability detection research. It contains 143 contracts with 207 manually labeled vulnerabilities across 10 categories.

Environment: macOS ARM64 (Apple Silicon M5 Pro), 48GB unified memory, Python 3.12. All tools executed locally with default configurations. External tools: Slither 0.9.6, Aderyn 0.6.x, Mythril 0.24.x, Solhint 5.0.x. LLM layers: Ollama with (16K context, temperature 0.1).

B. Results

TABLE II
Results on SmartBugs-curated (143 contracts, 207 ground-truth vulnerabilities)

Approach	Precision	Recall	F1
Slither (alone)*	8.3%	43.2%	13.9%
Mythril (alone)*	6.1%	27.4%	10.0%
Securify*	9.7%	36.8%	15.4%
SmartCheck*	4.2%	18.9%	6.9%
MIESC (static only)	9.3%	54.6%	15.9%
MIESC (all layers)	**22.7%**	**80.0%**	**35.4%**

*Baseline from Durieux et al. [5]

1) Overall Performance: Table II shows the results. With static analysis only (Layer 1: Slither + Aderyn), MIESC achieves 54.6% recall—already a 26% improvement over Slither alone. When all layers are enabled, recall reaches **80%**. The 25-percentage-point gain from adding Layers 3–9 is not uniform across categories: reentrancy and access control are already at 100% with Layer 1 alone (Table III), so the improvement comes entirely from categories that static analysis cannot cover—bad randomness (requires entropy analysis), front-running (requires transaction ordering reasoning), and arithmetic in pre-0.8 contracts (requires path-sensitive analysis). This demonstrates that the layers are *complementary*, not redundant.

The lower precision (9–23%) reflects informational findings (pragma version, naming conventions) that inflate the false positive count. For pre-audit triage, recall is the priority: missed vulnerabilities become production exploits costing millions, while false positives are filtered during manual review at negligible cost.

2) Category-Specific Detection: With static analysis (Layer 1), the framework achieves **100% recall** on access control, reentrancy, and the "other" category—the vulnerability classes most commonly exploited in DeFi. Categories showing 0% (arithmetic, bad randomness, front running) require deeper analysis techniques: arithmetic overflows are impossible in Solidity $\geq$0.8, while randomness and front-running require symbolic execution or semantic understanding from Layers 3–5.

TABLE III
Performance by Vulnerability Category (SmartBugs-curated)

Category	Precision	Recall	F1
Access Control	24.0%	**100%**	38.7%
Reentrancy	22.6%	**100%**	36.9%
Unchecked Low-Level Calls	6.9%	48.1%	12.1%
Denial of Service	2.6%	16.7%	4.4%
Other	14.3%	**100%**	25.0%
Time Manipulation	0%	0%	0%
Short Addresses	0%	0%	0%
Arithmetic	0%	0%	0%
Bad Randomness	0%	0%	0%
Front Running	0%	0%	0%

Static analysis only (Layer 1). Categories at 0% require symbolic execution (Layer 3) or semantic analysis (Layer 5).

3) Real-World Exploit Validation: To validate beyond academic benchmarks, we evaluated MIESC against 11 confirmed DeFi exploits totaling $3.3 billion in losses (Table IV). For each exploit, the original vulnerable contract source was analyzed and we checked whether MIESC flagged the specific vulnerability class that enabled the exploit.

TABLE IV
Detection of Real-World Exploits ($3.3B total losses)

Vulnerability	Exploits	Detected	Recall
Reentrancy	3	3	**100%**
Access Control	3	3	**100%**
Flash Loan	2	2	**100%**
Arithmetic	1	1	**100%**
Oracle Manipulation	1	0	0%
Governance Attack	1	0	0%
Overall	**11**	**9**	**81.8%**

MIESC achieved 81.8% recall on real exploits with a Cohen's κ of 0.773, indicating *substantial agreement* ($\kappa > 0.6$) between MIESC's automated analysis and the ground truth established by actual on-chain exploits. Notable detections include the Euler Finance reentrancy ($197M), Ronin Bridge access control ($624M), and Parity wallet freeze ($280M).

4) Execution Performance: Table V summarizes MIESC's execution speed. With parallel execution (4 workers), the framework processes 347 contracts per minute with zero analysis failures.

TABLE V
Execution Performance on SmartBugs-curated

Metric	Value
Contracts analyzed	143
Total execution time	148 seconds
Average time per contract	1.03 seconds
Parallel speedup (4 workers)	3.53×
Throughput (parallel)	347 contracts/minute
Success rate	100% (0 errors)

5) EVMBench: Business Logic Vulnerability Detection: To evaluate MIESC on vulnerabilities requiring semantic

understanding—business logic flaws, economic exploits, and protocol-level errors—we benchmark against EVMBench [15], a dataset of 40 real Code4rena audit contests containing 120 ground-truth vulnerabilities curated by OpenAI and Paradigm. Unlike SmartBugs' synthetic patterns, EVMBench vulnerabilities require reasoning about protocol invariants, value flows, and multi-contract interactions.

MIESC supports multiple analysis modes through a unified pipeline: the same Chain-of-Thought audit prompt, protocol-specific RAG enrichment, and finding normalization is applied regardless of provider, enabling fair comparison.

Matching methodology. To determine whether a MIESC finding matches an EVMBench ground-truth vulnerability, we apply a multi-signal matcher: (1) function name overlap, (2) semantic category matching across 10 vulnerability classes, (3) keyword overlap with stemming, (4) bigram matching, and (5) an LLM judge (Claude Haiku) as a final arbiter when signals 1–4 are inconclusive. This differs from EVMBench's official LLM judge protocol; we publish our matcher for reproducibility. Results exhibit variance across runs due to LLM non-determinism (typical range: ±8pp on 10 audits, ±4pp on 40 audits).

TABLE VI
EVMBENCH RECALL BY PROVIDER AND SCALE

Mode	Recall	Prec.	F1	Cost	n
Full benchmark (40 audits, 120 vulns):					
Static only	11.8%	—	—	$0	10[a]
+ GPT-4o	73.7%	20.0%	31.5%	$1.20	40
+ Claude Sonnet 4.6	82.5%	34.0%	48.2%	$5.50	40
Ablation — 10-audit subset (17 vulns):					
+ Ollama qwen2.5-coder:32b	82.4%	20.0%	32.2%	$0	10
+ Claude Sonnet 4.6	94.1%	25.0%	39.5%	$1.35	10
+ Claude Opus 4.6	88.2%	23.8%	37.5%	$4	10

[a]Static-only evaluated on 10-audit subset; recall on the full set is structurally identical (pattern matching does not depend on sample size). Ollama 40-audit validation pending (local model, $0 cost).

Table VI reveals four key findings. First, static analysis alone achieves only 11.8% recall on business logic vulnerabilities, confirming that pattern-based tools cannot detect semantic flaws. Second, a local 32-billion-parameter model (qwen2.5-coder:32b via Ollama) achieves 82.4% recall with *zero API cost and full data locality*—surpassing GPT-4o (64.7%) and approaching Claude Opus (88.2%). Third, Claude Sonnet 4.6 achieves 94.1% recall at $1.35 per audit, outperforming the larger Opus model (88.2% at $4)—suggesting that model scale is less important than prompt engineering and tool orchestration for security auditing. Fourth, all MIESC configurations substantially outperform Claude Opus as a standalone auditor (45.9% on the EVMBench leaderboard), demonstrating that **orchestrated static analysis combined with any LLM is strictly superior to even the best LLM alone**.

The practical implication is significant: a security researcher can run MIESC with a local model at zero cost and achieve recall competitive with commercial AI audit services, while a frontier API ($1.35/audit vs. $20K–60K for manual audits) pushes recall above 82%. We note that individual runs exhibit variance due to LLM non-determinism: Claude Sonnet ranged from 76% to 94% across runs on the 10-audit subset, stabilizing at 82.5% on the full 40-audit benchmark. All results reported are single-run unless noted.

V. DISCUSSION

A. Layer Contribution Analysis

The jump from 54.6% (static only) to 80% (all layers) raises the question: which layers contribute the missing 25 percentage points? Our per-category results provide the answer. Reentrancy and access control are fully covered by Layer 1 (Slither's detectors are mature for these classes). The gain comes from categories where static analysis is structurally limited: bad randomness requires reasoning about entropy sources (Layer 5 LLM analysis), front-running requires understanding transaction ordering semantics (Layer 5), and arithmetic overflows in pre-0.8 contracts need path-sensitive analysis (Layer 3 Mythril). This suggests the layers are genuinely complementary rather than redundant—a concern often raised about multi-tool approaches.

B. Comparison with SmartBugs 2.0

SmartBugs 2.0 [10] is the closest comparable framework, integrating 25 analysis tools with Docker-based execution. However, SmartBugs focuses on providing a *benchmarking platform* and does not publish detection metrics (precision, recall) on its own curated dataset. The tools it integrates (Mythril, Slither, Securify, Oyente, etc.) are individually benchmarked in Durieux et al. [5], where the best single tool achieves 43.2% recall (Slither).

A direct recall comparison is not possible because SmartBugs does not publish detection metrics. However, the individual tools it integrates achieve at most 43.2% recall independently [5]. The key differentiator of MIESC is not tool count but the AI/LLM layers (Layer 5) that detect vulnerability classes—logic flaws, semantic issues, protocol-specific risks—that static and symbolic analysis structurally cannot reach.

C. False Positive Management

Low precision (9–23%) is the primary usability concern. The raw finding count on SmartBugs-curated is 810 findings for 143 contracts (5.7 per contract), dominated by informational issues (pragma version, naming conventions). The FP filter addresses this through three mechanisms: (1) per-detector calibration based on empirical FP rates from our benchmark runs (e.g., Slither's detector has near-100% FP rate on modern contracts), (2) Solidity version awareness that suppresses integer overflow warnings for contracts using $\geq$0.8 (which has built-in overflow protection), and (3) library detection that recognizes OpenZeppelin and Solmate safe patterns. After filtering, the actionable finding count drops to 2–3 per contract on average. Since v5.2.0, an intelligence engine performs semantic deduplication, context-aware FP suppression, and cross-tool Bayesian confidence scoring, achieving approximately 30% noise reduction in practice. A trainable

FP classifier (GradientBoosting) is included but awaits a larger labeled auditor corpus for production deployment.

D. Practical Deployment

MIESC is designed for practical adoption:

- **GitHub Action**: One-line CI/CD integration with SARIF upload to GitHub Security tab.
- **Docker images**: Standard (3GB, multi-arch) and full (8GB, all tools) images on GHCR.
- **Plugin system**: PyPI-based entry points for community detectors.
- **Profiles**: Predefined configurations (fast, ci, defi, thorough) for different use cases.

E. Analysis of Missed Exploits

Two real-world exploits were not detected: the BonqDAO oracle manipulation ($120M) and the Beanstalk governance flash loan attack ($182M). Both require *semantic understanding* of protocol-specific logic that static pattern matching cannot capture. BonqDAO's vulnerability was in the interaction between a Tellor oracle and a custom stablecoin—detecting this requires modeling the oracle's trust assumptions, not just code patterns. Beanstalk's attack exploited governance voting with flash-borrowed tokens in a single transaction—a logical flaw invisible to pattern-based analysis. These findings motivate future work on protocol-aware semantic analysis in Layers 5–7.

F. Threats to Validity

Internal validity. The SmartBugs-curated ground truth was established by Durieux et al. [5] and may contain labeling errors. Our finding classifier maps MIESC tool outputs to SmartBugs categories using SWC IDs and keyword matching, which may introduce classification errors. We mitigate this by publishing the classifier source code for inspection.

External validity. The SmartBugs benchmark contains contracts from 2016–2020, which may not represent modern DeFi protocols. We address this with the real-world exploit evaluation (2020–2023 exploits), though the 11-exploit sample is small. The Etherscan distribution analysis (500 verified contracts) provides additional evidence of applicability to production code.

Construct validity. Recall is our primary metric because missed vulnerabilities have higher cost than false positives in pre-audit triage. However, precision remains important for usability—a tool that generates too many false positives will be abandoned by practitioners.

Reproducibility. All benchmark scripts, ground truth data, and evaluation methodology are included in the repository. Results can be reproduced with:

G. Limitations

- The Starknet/Cairo analyzer originally used line-by-line regex matching; as of v5.1.9, a block-level scanner (with a line-offset table for accurate source mapping) handles multi-line patterns such as followed by , closing the previously reported gap.
- Move (Sui/Aptos) and Solana/Anchor analyzers are scaffolded for community contribution but not yet evaluated against ground-truth benchmarks.
- Full 9-layer analysis requires significant resources (8GB+ Docker image, 16GB RAM).
- Tool availability varies by platform (Echidna/Medusa are amd64-only).
- LLM-based layers depend on model quality and introduce non-determinism; the multi-LLM consensus mechanism (Section III-D) mitigates but does not eliminate this; results may vary across Ollama model versions.
- The trainable shipped with the framework is deployed but dormant: activating it requires a labeled corpus of auditor-validated findings, which we are collecting as an ongoing effort.
- The real-world exploit evaluation (11 contracts) is limited in size; expanding to 100+ exploits and adding a Starknet-specific exploit corpus (zkLend, Braavos, Nostra) is planned.

VI. Conclusion

The central finding of this work is that no single analysis technique suffices for smart contract security, but *any* LLM combined with static analysis orchestration outperforms even the best LLM alone. On pattern-based vulnerabilities (SmartBugs), orchestrating multiple static tools raises recall from 43% (Slither alone) to 80%. On business-logic vulnerabilities (EVMBench), static analysis alone achieves only 11.8%—but combining it with a local 32B-parameter model reaches 82.4% at zero cost, and 94.1% with Claude Sonnet at $1.35 per audit. Both configurations substantially outperform Claude Opus as a standalone auditor (45.9% on the leaderboard). This demonstrates that **orchestrated static analysis + LLM is strictly superior to LLM alone**: every MIESC configuration—including a free local model—outperforms Claude Opus standalone (45.9%) by at least 18 percentage points. On the 10-audit subset, the local model (82.4%) outperforms GPT-4o (64.7%), suggesting that orchestration quality matters at least as much as model scale, though 40-audit validation of the local model is needed to confirm this at scale.

MIESC's practical contribution is making this accessible: runs Slither, Aderyn, an intelligence engine with Bayesian cross-tool confidence, protocol-specific RAG enrichment, and a local LLM audit—entirely free, offline, and DPGA-compliant. Adding a frontier API (, $1.35/audit) pushes recall to 94%, compared with $20K–60K for a manual audit engagement.

The framework's agentic and formal-verification components are designed around the same observation: an isolated LLM verdict or a standalone prover run is rarely sufficient, but combining them—with multi-LLM consensus on critical findings, and with automatically generated specifications that feed directly into provers—tightens the loop between detection and proof. The 2024–2026 Cairo exploit coverage extends this

principle to a second ecosystem, encoding concrete lessons from zkLend ($9.6M) and Braavos into machine-checkable patterns.

Three directions for future work emerge from these results. First, precision needs improvement—the current 2–3 actionable findings per contract after filtering is usable, but the trainable FP classifier included in MIESC is waiting on a labeled auditor corpus to reduce this further. Second, the 11-exploit evaluation should expand to 100+ contracts (including a Starknet/Cairo exploit corpus) to strengthen statistical claims. Third, MIESC v5.3.0 introduced for automated text-level remediation—inserting modifiers, access-control guards, and wrapping—with a 100% hit rate on test contracts, alongside for mapping findings to 12 international standards. Generating Foundry test harnesses from exploit scenarios to formally verify that each fix discharges the original CVL rule remains future work.

MIESC is available under AGPL-3.0 (see *Data Availability*).

ACKNOWLEDGMENT

This work originated from a Master's thesis in Cyberdefense at Universidad de la Defensa Nacional (UNDEF) – IUA Córdoba, Argentina, and continues as a research project at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The author thanks the Ethereum security community and the developers of Slither, Mythril, Echidna, Foundry, Aderyn, and Halmos for their foundational contributions.

Data Availability

MIESC source code, benchmark scripts, ground truth datasets, and evaluation results are publicly available at https://github.com/fboiero/MIESC under AGPL-3.0. The SmartBugs-curated dataset is from Durieux et al. [5]. The real-world exploit dataset was curated from DeFiHackLabs (https://github.com/SunWeb3Sec/DeFiHackLabs).

REFERENCES

[1] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.

[2] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

[3] B. Mueller, "Smashing Ethereum smart contracts for fun and real profit," in *HITB Security Conference*, 2018.

[4] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, "Echidna: Effective, usable, and fast fuzzing for smart contracts," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.

[5] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 530–541.

[6] Cyfrin, "Aderyn: Rust-based Solidity ast analyzer," https://github.com/Cyfrin/aderyn, 2024.

[7] a16z crypto, "Halmos: Symbolic testing for EVM smart contracts," https://github.com/a16z/halmos, 2023.

[8] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A user-friendly symbolic execution framework for binaries and smart contracts," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1186–1189.

[9] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze Solidity smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.

[10] M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.

[11] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

[12] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[13] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[14] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io, 2024.

[15] A. Gupta, V. Gottimukkala, D. Robinson, Paradigm, and OpenAI, "EVMBench: Evaluating AI agents on smart contract security," *arXiv preprint arXiv:2603.04915*, 2026. [Online]. Available: https://github.com/paradigmxyz/evmbench